

Workflow Enforcement & Handoff

บังคับลำดับงานด้วยโค้ด และส่งต่องานให้มนุษย์อย่างครบถ้วน

Domain 1 · Task 1.4 — Claude Certified Architect

แก่นของ Task นี้

"Prompt-based guidance is **probabilistic** — it works **most** of the time.
Programmatic enforcement is **deterministic** — it works **every** time."

Prompt = probabilistic

สั่งด้วย prompt → ได้ผล "เกือบทุกครั้ง"

Code = deterministic

บังคับด้วยโค้ด → ได้ผล "ทุกครั้ง"

ข้อสอบทดสอบว่าคุณรู้ว่า **เมื่อไหร่ prompt ไม่พอ** และต้องเลื่อนไปใช้การบังคับระดับโค้ด

The Enforcement Spectrum

Prompt-based guidance

ฝัง instruction ลงใน system prompt — เช่นสั่งให้ agent ตรวจสอบตัวตนก่อนคืนเงิน

90–95% success

ยังมีโอกาสพลาดโดยเนื้อแท้ (inherent failure risk)

Programmatic enforcement

ใช้ **code-level gates** ที่ "บล็อก" ลำดับการทำงานที่ผิดในทางกายภาพ

100% reliable

เป็นกำแพงทางเทคนิค ไม่ใช่การขอร้องเชิงพฤติกรรม



Key Concept: Decision Rule

เลือกวิธีบังคับตาม "**ความเสี่ยงของงาน**" ไม่ใช่ตามความสะดวก

✓ Programmatic enforcement — งานเดิมพันสูง

Financial — refunds, transfers

Security — identity verification, access control

Compliance — regulatory, AML checks

⚡ Prompt guidance ได้แก่งานเดิมพันต่ำ

รูปแบบการจัดหน้า (formatting)

สไตล์การเขียน (style)

Prerequisite Gates

คือการตรวจสอบเชิงโปรแกรมที่ **"บล็อกการเรียก tool"** จนกว่าเงื่อนไขก่อนหน้าจะครบ ไม่ใช่แค่เตือนใน prompt — แต่ทำให้ทำผิดลำดับไม่ได้จริง

ตัวอย่าง

ถ้าตัวตนลูกค้ายังไม่ถูกยืนยัน การเรียก `process_refund` จะคืน error กลับมา:

```
"Cannot process refund – customer identity not verified."
```

แทนที่จะทำงานต่อ → นี่คือรูปธรรมของ **deterministic enforcement**

Lifecycle Hooks

SubagentStart ยิงตอน subagent ถูก **spawn** ขึ้นมา — ใช้ทำ **rate limiting** และ validation ก่อนเริ่มงาน

SubagentStop ยิงตอน subagent ทำงาน**เสร็จ** — ใช้ intercept output เพื่อ validate ผลลัพธ์ และ strip ข้อมูลที่ไม่ควรหลุด

Pre/PostToolUse subagent กำหนด hook แบบ **scoped** ได้ — มีผลเฉพาะภายในการทำงานของ subagent ตัวนั้น ไม่กระทบตัวอื่น

Multi-Concern Handling

สถานการณ์: คำขอเดียวมีหลายประเด็นปน — เช่น ลูกคำถามเรื่องคืนเงิน + เปลี่ยนแผน + ร้องเรียนพร้อมกัน

1. แยกคำขอ

แยกเป็นรายการย่อยที่ชัดเจน

2. สืบสวน parallel

แต่ละประเด็นพร้อมกัน ใช้ **shared context**

3. สังเคราะห์

รวมเป็นการแก้ปัญหาขึ้นเดียวครบทุกประเด็น

× สิ่งที่ผิด

จัดการแบบเรียงทีละอัน (sequential) หรือตอบแค่บางประเด็น (partial handling)

Structured Handoff Protocol

★ ออกสอบบ่อย

มนุษย์ผู้รับงาน **"ไม่เห็น"** conversation transcript ของ agent
จึงต้องส่งมอบข้อมูลอย่างมีโครงสร้างครบ **5 องค์ประกอบ**

1

customer ID

2

conversation
summary

3

root cause analysis

4

refund amount

5

recommended action

ถ้าขาดฟิลด์สำคัญไป → การส่งต่อ**ไม่สมบูรณ์** และทำให้ประสบการณ์ลูกค้าแย่

⚠ Exam Traps ที่ต้องระวัง

ตัวหลอก #1 — "prompt ที่เขียนดีขึ้น" = deterministic

× ไม่มีทาง — prompt เก่งแค่ไหนก็ยัง **probabilistic**

ตัวหลอก #2 — few-shot examples garanti 100%

× ช่วยให้ดีขึ้นแต่ไม่ "รับประกัน" การบังคับครบทุกครั้ง

ตัวหลอก #3 — ใช้ routing classifier แก่ workflow enforcement

× classifier แก่เรื่อง **routing** ไม่ได้บังคับลำดับงานภายใน agent sequence

ตัวหลอก #4 — handoff summary แบบขาดฟิลด์

× เช่นลืม root cause หรือ recommended action → ประสิทธิภาพลูกค้าแย่



Practice Scenario

agent คืนเงินถูกสั่งใน system prompt ให้ "**ยืนยันตัวตนลูกค้าก่อน process refund เสมอ**" แต่บันทึกพบว่า ~**5-8%** ของเคส agent ยังคืนเงินโดยไม่ยืนยันตัวตน ทีมต้องการความถูกต้อง **100%** เพราะเป็น financial operation — ควรแก้ไขอย่างไร?

- A. เขียน system prompt ใหม่ให้หนักแน่นและชัดเจนกว่าเดิม เน้นย้ำเรื่องยืนยันตัวตน
- B. เพิ่ม few-shot examples ของเคสที่ยืนยันตัวตนถูกต้องให้ agent เรียนรู้มากขึ้น
- C. ใส่ routing classifier เพื่อจัดเส้นทางเคสคืนเงินให้แม่นยำขึ้นก่อนถึง agent
- D. ใส่ prerequisite gate เชิงโปรแกรมให้ process_refund คืน error ถ้าตัวตนยังไม่ถูกยืนยัน



หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option D

financial operation ที่ต้องการ 100% ต้องใช้ **programmatic enforcement** — prerequisite gate ที่บล็อก process_refund จนกว่าตัวตนจะถูกยืนยัน ทำให้การทำผิดลำดับเป็นไปไม่ได้จริง (deterministic)

A และ B — ยังเป็น prompt-based ที่ probabilistic → prompt ที่ดีขึ้นหรือ few-shot ช่วยได้แต่ไม่การันตี 100% จึงยังหลุด 5-8% อยู่ดี

C — เอา routing มาแก้ผิดปัญหา เพราะ classifier จัดการแค่การเลือกเส้นทาง ไม่ได้บังคับลำดับงานภายใน agent

สรุป Task 1.4

⚖️ prompt = **probabilistic** · code = **deterministic**

🔒 งานเดิมพันสูง (การเงิน/ความปลอดภัย/compliance) → **prerequisite gates**

🔗 Lifecycle hooks: SubagentStart / SubagentStop

📋 Structured handoff ควบ 5 พาด — มนุษย์ไม่เห็น transcript

⚠️ trap: prompt เก่ง / few-shot / routing classifier ไม่ใช้การบังคับ

🎯 จำเส้นแบ่ง probabilistic/deterministic ให้ขึ้นใจ



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

ข้อมูล production พบว่า 8% ของเคส customer support agent ทำ refund โดยไม่ได้ตรวจสอบความเป็นเจ้าของบัญชี บางครั้งจึงคืนเงินผิดพลาด ซึ่งที่ system prompt ระบุว่า "always verify customer identity before processing refunds" — วิธีแก้ที่เหมาะสมที่สุดคืออะไร?

A. ใส่ programmatic prerequisite gate ที่บล็อก `process_refund` จนกว่า `get_customer` จะคืน customer ID ที่ verified แล้ว

B. เพิ่ม instruction ที่หนักแน่นขึ้นใน system prompt เน้นย้ำความสำคัญของการ verify ก่อนทำ refund

C. เพิ่ม few-shot examples ที่สาธิตลำดับ verification-then-refund ที่ถูกต้อง

D. ใส่ routing classifier ที่ส่งคำขอ refund ทั้งหมดเข้า pipeline แบบ verification-first เฉพาะทาง

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option A

financial operation ต้องการการบังคับแบบ deterministic — prerequisite gate จะป้องกันไม่ให้ refund tool ทำงานทางกายภาพจนกว่าการ verify ตัวตนจะเสร็จ จึงจำกัด failure rate 8% ได้หมด เป็นทางเดียวที่การันตี 100%

ทำไมข้ออื่นผิด

B — prompt ปัจจุบันสั่งให้ verify อยู่แล้วแต่ยังพลาด 8% การเขียน prompt แรงขึ้นอาจลดเหลือ 3-4% แต่จำกัดไม่ได้ financial ต้องการ deterministic ไม่ใช่ probabilistic

C — few-shot ช่วยเรื่อง consistency แต่ยังมี failure rate ไม่เป็นศูนย์ สำหรับ financial ที่พลาดครั้งเดียวก็คือคืนเงินผิดบัญชี การปรับแบบ probabilistic ไม่พอ

D — routing classifier จัดการ "คำขอไปถึง agent อย่างไร" ไม่ใช่ "agent execute workflow ภายในอย่างไร" ปัญหาคือ agent ข้าม verify ภายในการทำงานของตัวเอง ต้องใช้กลไก enforcement ระดับ per-agent



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

เมื่อ agent escalate งานไปหา human agent ทำไม handoff summary ต้องมีฟิลด์ที่เฉพาะเจาะจง?

- A. human agent ชอบ structured data มากกว่า conversational summary
- B. human agent **ไม่มี** access ไปดู conversation transcript จึงต้องการ summary ที่ self-contained
- C. structured handoff เป็นข้อกำหนดของ Claude API specification
- D. monitoring system ต้องการฟิลด์เฉพาะเพื่อทำ compliance tracking

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option B

ข้อจำกัดสำคัญคือ human agent ไม่สามารถเลื่อนดู chat history ได้ handoff summary จึงเป็นข้อมูลเดียวที่เขาได้รับ ถ้าไม่มี summary ที่ครบและ self-contained (customer ID, conversation summary, root cause, refund amount, recommended action) human agent ต้องให้ลูกค้าเล่าใหม่ทั้งหมด

ทำไมข้ออื่นผิด

A — ไม่ใช่เรื่องความชอบ (preference) แต่เป็นเพราะ human agent ไม่มี access ไปดู conversation ก่อนหน้าจริง ๆ

C — รูปแบบ handoff ไม่ใช่ข้อกำหนดของ API specification แต่เป็น architectural design decision ที่ตั้งบนความจริงว่า human ไม่เห็น conversation

D — แม้ monitoring จะได้ประโยชน์จาก structured data แต่เหตุผลหลักคือการที่ human agent เข้าถึง transcript ไม่ได้



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

ทีม compliance กำหนดว่า workflow step หนึ่งต้องเกิดขึ้น 100% ของเวลา ก่อน financial operation จะทำงาน — วิธีใดให้การรับประกันนี้ได้?

- A. ใส่ข้อกำหนดใน system prompt ด้วยตัวหนาและการเน้นย้ำซ้ำ ๆ
- B. ใช้ few-shot examples ที่สาธิตลำดับที่ถูกต้องใน 10 สถานการณ์ต่างกัน
- C. ใส่ hook หรือ prerequisite gate ที่บล็อก operation เชิงโปรแกรมจนกว่า step นั้นจะเสร็จ
- D. เพิ่ม validation agent แยกที่ตรวจ compliance ก่อน forward ไปยัง financial agent

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option C

hook และ prerequisite gate ให้ enforcement แบบ deterministic — operation ไม่สามารถทำงานทางกายภาพ ได้จนกว่า prerequisite จะเสร็จ นี่คือกลไกเดียวที่การันตี compliance 100%

ทำไมข้ออื่นผิด

- A** — instruction ใน prompt ไม่ว่าจะจัดรูปแบบหรือเน้นย้ำแค่ไหนก็ยัง probabilistic ช่วยเพิ่ม rate แต่การันตี 100% ไม่ได้ พลาดครั้งเดียวใน financial ก็มีผลจริง
- B** — few-shot ช่วยเพิ่ม accuracy แต่ยัง probabilistic 10 ตัวอย่างอาจได้ 98% แต่ 100% ต้องใช้กลไก deterministic
- D** — validation agent เองก็ probabilistic อาจพลาดจับ violation เป็นบางครั้ง มีเพียง programmatic gate เท่านั้นที่การันตีแบบ deterministic



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

ลูกค้าส่งคำขอที่มี 3 ประเด็น: คืนสินค้า (return an order), ร้องเรียนค่าธรรมเนียม (dispute a charge) และแก้ที่อยู่ (update address) — agent ควรจัดการอย่างไร?

- A. จัดการประเด็นที่เร่งด่วนที่สุดก่อน แล้วให้ลูกค้าโทรกลับมาใหม่สำหรับอีก 2 ประเด็น
- B. แยกเป็น 3 รายการย่อยที่ชัดเจน สืบสวนแบบ parallel โดยใช้ shared context แล้วสังเคราะห์เป็นการแก้ปัญหาขึ้นเดียวที่ครบ
- C. จัดการแต่ละประเด็นในบทสนทนาแยกกันเพื่อเลี่ยงความสับสน
- D. ส่งทั้ง 3 ประเด็นให้ human agent เพราะ compound request ซับซ้อนเกินไป

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

คำขอหลายประเด็นควรถูก decompose เป็นรายการย่อยที่ชัดเจน สืบสวนแบบ parallel โดยใช้ shared context (ข้อมูลบัญชีลูกค้าเกี่ยวข้องกับทั้ง 3 ประเด็น) แล้วสังเคราะห์เป็นคำตอบเดียวที่ครอบคลุมทุกประเด็น

ทำไมข้ออื่นผิด

- A — ไม่ได้แก้ทุกประเด็น และสร้างประสบการณ์ที่แย่โดยบังคับให้ลูกค้าโทรกลับมาใหม่
- C — บทสนทนาแยกกันทำให้เสีย shared context (ข้อมูลบัญชีลูกค้า) และบังคับลูกค้าผ่านหลายรอบการโต้ตอบ
- D — compound request เป็นเรื่องปกติของงาน support การ escalate ทั้งหมดให้มนุษย์ทำลายจุดประสงค์ของ agent



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

สถานการณ์ใดที่ใช้ prompt-based guidance (แทน programmatic enforcement) เป็นแนวทางที่ยอมรับได้?

- A. การรับประกันว่า refund จะถูก process หลังจาก verify ตัวตนแล้วเท่านั้น
- B. การบังคับให้ทำ anti-money laundering (AML) check ก่อน international transfer
- C. การจัดรูปแบบคำตอบของ agent เป็น markdown ที่มี header และ bullet point
- D. การบล็อกการลบบัญชี (account deletion) โดยไม่มี manager approval

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option C

formatting preference เป็นงาน low-stakes การตอบเป็น plain text แทน markdown เป็นครั้งคราวไม่ใช่ความเสี่ยงทางธุรกิจ prompt-based guidance จึงเพียงพอสำหรับข้อกำหนดด้าน style และ formatting

ทำไมข้ออื่นผิด

A — refund โดยไม่ verify เป็น financial risk ต้องใช้ programmatic enforcement

B — AML check เป็นข้อกำหนดด้าน regulatory การพลาดครั้งเดียวอาจเจอบทลงโทษทางกฎหมาย ต้องบังคับแบบ deterministic ด้วย hook

D — account deletion เป็นงาน high-stakes ที่ irreversible ต้องใช้ programmatic enforcement (require manager approval token) เพื่อการันตีขั้นตอน approval